

AEGISOPS AI

AI-Powered Incident Management, Observability & SRE Platform

Complete Project Understanding & Solo-Developer Blueprint

Purpose of this document

This is not only a feature list. It explains what the product is, why it exists, who uses it, how data moves through it, where the AI is real, what each technology contributes, how a solo software-engineering student can build it, how to deploy it live, how to demonstrate it, and how to defend the engineering decisions in an interview.

Prepared for Zeeshan
BS Software Engineering - Final Semester
September 2026

Contents

1. Executive Explanation
2. The Real-World Problem
3. What AegisOps AI Is - and Is Not
4. Target Users and Roles
5. End-to-End Product Story
6. Core Feature Modules
7. AI Capabilities and Guardrails
8. System Architecture
9. Technology Stack and Why Each Choice Exists
10. Data and Database Design
11. API and Real-Time Design
12. Observability and Telemetry Ingestion
13. Incident Detection Logic
14. Root-Cause Assistance and AI Reasoning
15. Security, RBAC and Multi-Tenancy
16. Background Jobs, Notifications and Reliability
17. Testing Strategy
18. Deployment and Live Demo Strategy
19. MVP vs Advanced Scope
20. Solo-Developer Build Roadmap
21. How This Benefits a Software Engineering Degree
22. CV, GitHub and Interview Value
23. How to Demonstrate the Project
24. Risks, Limitations and Ethical Use
25. Antigravity Prompt Strategy
26. Definition of Done
27. Example Interview Questions
28. Glossary
29. Reference Sources

1. Executive Explanation

AegisOps AI is a production-style platform that helps software teams understand when a running application is becoming unhealthy, turn technical signals into structured incidents, coordinate the response, and use AI to summarize evidence and suggest likely investigation paths. It combines software engineering, backend systems, real-time communication, databases, observability, DevOps and AI in one coherent product.

One-sentence explanation

"AegisOps AI watches software-system signals, detects abnormal behavior, creates and manages incidents, and gives engineers evidence-grounded AI assistance for triage and root-cause investigation."

How you explain it to a non-technical person

Imagine an online store suddenly becomes slow and customers cannot complete checkout. Many technical systems may be involved: the web server, API, database, cache, payment service or network. AegisOps AI acts like a control room. It collects health signals, detects that something is wrong, creates an incident, shows what changed, and gives the engineering team a concise AI-generated explanation of what to inspect first.

How you explain it to a software engineer

It is an incident-management and observability application with telemetry ingestion, metric-based rules, anomaly detection, incident correlation, real-time dashboards, RBAC, alert workflows, background processing, audit trails and an AI analyst that operates on retrieved incident evidence instead of inventing unsupported answers.

Why this is a strong portfolio project

A basic student app proves that you can create screens and CRUD endpoints. AegisOps AI can prove that you understand system boundaries, failure modes, asynchronous jobs, streaming or event-driven concepts, data modeling, authorization, observability, reliability, testing and safe AI integration. Those are software-engineering concerns, not only coding concerns.

2. The Real-World Problem

Modern applications produce large amounts of operational data. A single problem can produce CPU spikes, increased response times, HTTP errors, failed jobs and unusual logs at the same time. If every signal is treated independently, engineers receive noise rather than understanding.

Operational problem	What happens without a good platform	What AegisOps AI adds
Too many alerts	Engineers receive duplicate or low-value notifications.	Grouping, deduplication, severity and incident correlation.
Slow triage	Someone manually opens dashboards and logs to understand the symptom.	Single incident workspace with related evidence.
Knowledge gap	Junior engineers may not know what to check first.	AI-generated investigation hypotheses and next checks.
Poor coordination	No clear owner, status or incident timeline.	Assignment, acknowledgement, status, notes and timeline.
No learning loop	After resolution the team forgets why it happened.	Postmortem, root-cause record and resolution knowledge.
Weak visibility	Managers cannot see reliability trends.	Service health, incident frequency, MTTA/MTTR and trend dashboards.

The key engineering insight

The project is not "make a dashboard that displays CPU." The important problem is turning raw technical signals into a reliable workflow: observe -> detect -> correlate -> triage -> assign -> investigate -> resolve -> learn.

3. What AegisOps AI Is - and Is Not

It IS	It is NOT
A real incident-management workflow.	A static dashboard with fake charts only.
An observability learning platform that ingests telemetry.	A replacement for every enterprise monitoring product.
A safe AI assistant grounded in incident evidence.	An AI that claims a root cause with no evidence.
A deployable portfolio SaaS-style application.	A tutorial project with one route and one database table.
A system that can integrate with demo services and real APIs.	A requirement to own a huge production infrastructure.
A platform demonstrating SRE concepts.	A fully autonomous production remediation system.

For the portfolio version, you will build a realistic product boundary. You will demonstrate the complete lifecycle with a controllable demo environment and optional integrations. This gives you credible engineering depth without pretending to reproduce the scale of commercial tools used by global infrastructure teams.

4. Target Users and Roles

Role	Primary responsibilities inside the product
Organization Owner / Admin	Create organization, configure services, invite members, manage roles, integrations and policies.
SRE / DevOps Engineer	Monitor services, acknowledge incidents, investigate telemetry, manage alert rules and resolve incidents.
Software Engineer	View incidents affecting owned services, add notes, inspect traces/log evidence and collaborate on resolution.
Team Lead / Engineering Manager	Track reliability trends, incident ownership, MTTA/MTTR, recurring failure areas and postmortems.
Read-Only Viewer	View dashboards and incident status without modifying operational state.

Why roles matter

RBAC is a core software-engineering feature. Real systems should not allow every signed-in person to modify alert rules, delete services or close incidents. Your implementation should check permissions on the backend, not only hide buttons in the frontend.

5. End-to-End Product Story

A useful way to understand the entire platform is to follow one incident from beginning to end.

1. A demo checkout API is registered in AegisOps AI as a monitored service.
2. The demo service exports metrics such as request rate, error rate and response latency. It can also emit traces and structured logs.
3. AegisOps telemetry adapter collects or receives the data and stores the operational information needed by the platform.
4. A rule detects that the 5xx error rate is above a threshold, or the anomaly detector identifies a statistically unusual pattern.
5. The system creates an alert event. Correlation logic checks whether similar alerts belong to an existing open incident.

6. If needed, AegisOps creates a new incident with severity, affected service, first-seen timestamp and attached evidence.
7. The browser receives a real-time update. The incident appears on the live dashboard without refreshing the page.
8. An engineer opens the incident and sees symptoms, recent changes, relevant metric windows, related logs and a timeline.
9. The AI analyst receives a structured evidence package and produces a summary, possible hypotheses, confidence labels and recommended next checks.
10. The engineer acknowledges the incident, assigns ownership, adds investigation notes and updates status.
11. When the fault is fixed, the engineer resolves the incident and records the confirmed cause and resolution.
12. A postmortem can be generated from the timeline, but the engineer reviews and edits it before finalizing.
13. The analytics page updates reliability statistics and recurring incident patterns.

Incident Lifecycle



AegisOps AI stores the complete timeline so an interviewer can see detection, decision, collaboration and resolution - not only a chart.

6. Core Feature Modules

Module	Professional functionality
Authentication & Organization	Sign up/sign in, organization workspace, member invitations, secure sessions, password management, account lock protections.
RBAC & Permissions	Owner/Admin/Engineer/Viewer roles, permission guards, backend authorization, audit of privileged actions.
Service Catalog	Register services, owner/team, environment, endpoints, tags, dependencies and health configuration.
Telemetry Ingestion	Receive metrics/events from demo agents or OpenTelemetry-compatible sources; validate and normalize incoming data.
Live Health Dashboard	Service status, request/error/latency summaries, active incidents, recent alerts and health trends.
Alert Rules	Threshold rules, evaluation windows, severity, cooldowns, enable/disable, preview and test rule functionality.
Anomaly Detection	Unsupervised statistical/ML model for unusual metric behavior with anomaly score and evidence window.
Incident Correlation	Group related alerts by service, time window, fingerprint and signal characteristics.
Incident Workspace	Severity, status, assignee, timeline, evidence, notes, related alerts, AI analysis, activity history.
AI Incident Analyst	Evidence-grounded summary, likely causes as hypotheses, next checks, impact explanation and postmortem assistance.
Notifications	In-app notifications; optional email/Slack/Discord webhook integration; deduplication and retry behavior.

Module	Professional functionality
Postmortems	Incident summary, impact, timeline, confirmed root cause, contributing factors, resolution, follow-ups.
Analytics	MTTA, MTTR, incidents by service/severity, recurrence, alert volume, top unstable services.
Audit Log	Who changed roles, alert rules, services, incident status, configuration and other sensitive data.
Settings & Integrations	Organization settings, AI provider configuration, telemetry keys, notification targets and retention settings.

7. AI Capabilities and Guardrails

AI Feature 1 - Metric Anomaly Detection

The first AI component should not depend on an LLM. A small unsupervised model such as Isolation Forest can learn what a recent metric window normally looks like and assign an anomaly score to unusual behavior. For example, a simultaneous jump in latency and error rate can be marked as abnormal even when a fixed threshold is not crossed.

For a portfolio implementation, begin with engineered features such as rolling mean, rolling standard deviation, rate of change, error ratio and latency percentiles. The model returns an anomaly score and the platform stores the exact data window used for the decision.

AI Feature 2 - Incident Summarization

The platform converts incident evidence into a concise summary: affected service, first symptom, strongest signals, time relationship and current impact. The LLM is not asked to invent facts; it is given a structured context payload retrieved from your own incident records.

AI Feature 3 - Root-Cause Assistance

The AI produces ranked hypotheses, not a definitive diagnosis. Example: "Hypothesis: database saturation may be contributing because API latency increased at the same time as database connection wait time." Every hypothesis should reference the evidence that caused it to be proposed.

AI Feature 4 - Next-Best Investigation Steps

The assistant can recommend checks such as inspect database connections, compare with the latest deployment, inspect a trace with the slowest span, or review errors around the incident start time. This makes the AI useful for operational reasoning rather than conversational decoration.

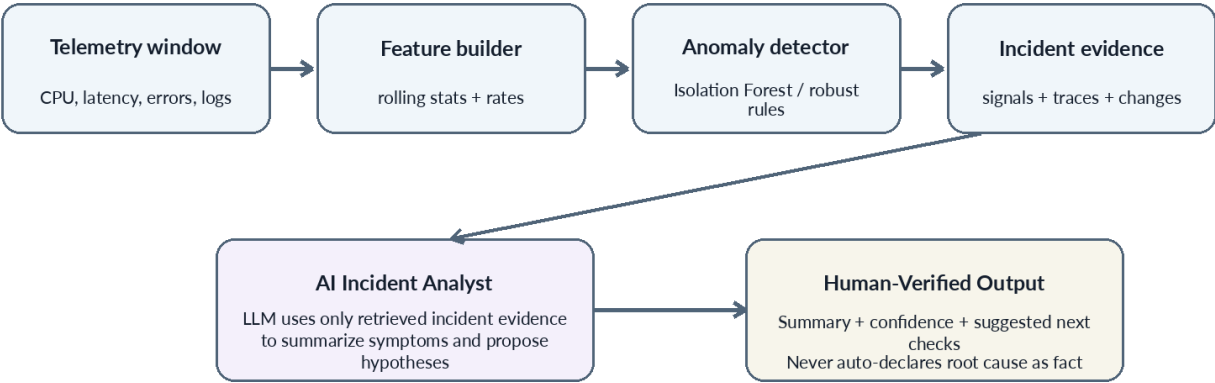
AI Feature 5 - Postmortem Drafting

After the incident is resolved, the AI can draft a postmortem from the stored timeline. The confirmed root cause must still come from the engineer. The system clearly distinguishes "AI-generated draft" from "human-confirmed cause."

AI guardrails

- Do not present model output as certainty.
- Show confidence or evidence strength where practical.
- Never allow the LLM to change production state directly in the portfolio version.
- Log AI requests and outputs for debugging and audit, while avoiding secrets.
- Redact credentials, tokens and sensitive log fields before AI processing.
- Use provider abstraction so the app can work with a cloud LLM or an optional local model.
- Require human confirmation for root cause and postmortem finalization.

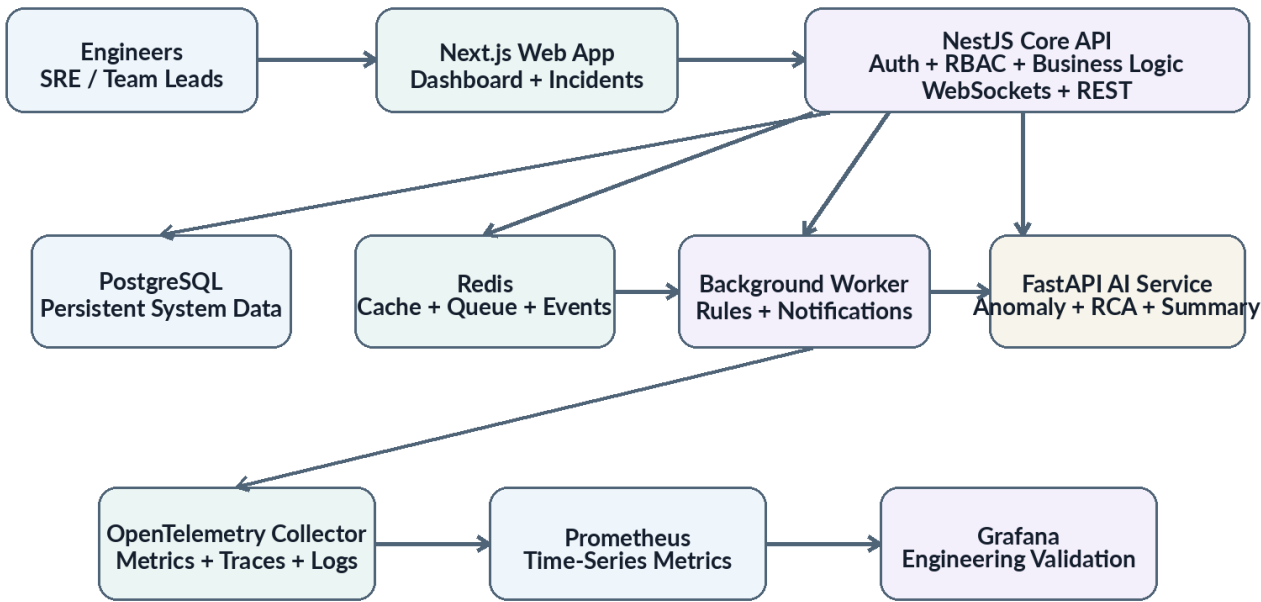
AI Decision Pipeline



8. System Architecture

The architecture is deliberately professional but solo-friendly. The core application remains a modular backend so you do not create distributed-system complexity just for appearance. AI is separated because Python is better suited to ML workloads, and background processing is separated so slow jobs do not block API requests.

AegisOps AI - Portfolio Architecture



Design principle: modular core backend + separate AI service + worker. Enterprise concepts without unnecessary solo-project complexity.

Major components

Component	Responsibility	Why separate it
Next.js web application	Dashboard, service pages, incidents,	Strong full-stack React ecosystem and

Component	Responsibility	Why separate it
	settings, charts, user interaction.	production deployment options.
NestJS core API	Auth, organizations, RBAC, service catalog, alerts, incidents, APIs, WebSockets.	Central business rules stay coherent and testable.
Background worker	Rule evaluation, notification delivery, scheduled calculations, retries.	Long-running work should not block request/response latency.
FastAPI AI service	Feature processing, anomaly scoring, incident reasoning interface.	Python ML ecosystem stays isolated behind a clean API.
PostgreSQL	Users, organizations, services, alert definitions, incidents, audit records.	Reliable relational modeling and transactional consistency.
Redis	Queue backing, caching, event fan-out, rate-limiting support.	Fast transient coordination outside the primary database.
Telemetry stack	OpenTelemetry Collector, Prometheus and optional Grafana.	Lets you learn industry observability concepts and validate your platform.

Why not start with many microservices?

A solo project with ten microservices often becomes a deployment exercise rather than a good product. It creates network failure modes, repeated authentication logic, distributed tracing requirements and operational overhead before you have enough load to justify them. A modular monolith demonstrates cleaner engineering judgment. If the project grows, selected modules can later be extracted.

9. Technology Stack and Why Each Choice Exists

Technology	Reason in this project
TypeScript	Shared language for frontend and core backend; strong typing improves refactoring and API contracts.
Next.js + React	Professional web UI, routing, server/client rendering options and strong ecosystem.
Tailwind CSS + shadcn/ui	Fast, consistent UI construction while keeping components customizable.
TanStack Query	Server-state caching, invalidation, loading/error states and efficient API-driven UI.
NestJS	Structured Node.js backend with dependency injection, guards, validation, WebSockets, queues and testing conventions.
Fastify adapter (optional)	Efficient HTTP runtime while retaining NestJS architecture.
PostgreSQL	Relational integrity for users, organizations, incidents, roles and configuration.
Prisma	Typed database access, migrations and schema-driven development.
Redis	Transient cache, job queues, distributed events and rate-limiting primitives.
BullMQ	Reliable Node.js background jobs with retries, delays and queue workers.
Python + FastAPI	ML/AI service boundary with simple typed HTTP APIs and Python ecosystem access.
scikit-learn	Practical anomaly-model baseline without requiring expensive model training infrastructure.
LLM provider adapter	Lets AI reasoning work with a selected provider and avoids coupling your business logic to one vendor.
OpenTelemetry	Vendor-neutral telemetry instrumentation for traces, metrics and logs.
Prometheus	Time-series metrics collection and alerting ecosystem.
Grafana (engineering validation)	Optional external view to validate that the telemetry your app sees is real.
Docker + Docker Compose	Reproducible multi-container local environment and simpler deployment story.
GitHub Actions	Automated lint, typecheck, tests, build and optionally deployment.
Vitest/Jest	Unit and integration tests for application and backend logic.

Technology	Reason in this project
Playwright	Browser-level end-to-end verification of critical workflows.

Important interview principle

Do not answer "I used Redis because it is fast" or "I used AI because it is trending." Each technology should map to a concrete system responsibility and trade-off.

10. Data and Database Design

The database represents the business truth of the platform. Telemetry can be high-volume, so the portfolio design should avoid blindly putting every raw time-series sample into the same transactional tables as users and incidents. Store durable product entities in PostgreSQL, while metric time series can remain in Prometheus or a bounded telemetry store.

Entity	Purpose
Organization	Tenant/workspace boundary.
User	Identity and profile.
Membership	User role inside organization.
Team	Optional ownership group.
Service	Monitored software service.
ServiceEnvironment	Production/staging/dev instance configuration.
TelemetrySource	API key / collector source metadata.
AlertRule	Threshold or detection policy.
AlertEvent	A specific rule/anomaly firing.
Incident	Correlated operational problem.
IncidentAlert	Many-to-many link between incident and alert events.
IncidentTimelineEvent	Status, assignment, note, AI analysis, resolution events.
IncidentEvidence	Metric/log/trace references or summarized evidence.
AIFinding	AI summary, hypotheses, confidence and evidence references.
NotificationTarget	Email/webhook/chat destination.
NotificationDelivery	Delivery state, retry count and failure detail.
Postmortem	Human-reviewed incident learning record.
AuditEvent	Security and configuration activity.

Important database relationships

- Organization 1 -> many memberships, services, alert rules, incidents and audit events.
- Service 1 -> many alert rules, alert events and incidents.
- Incident many <-> many alert events through IncidentAlert.
- Incident 1 -> many timeline events, evidence records and AI findings.
- User can own/acknowledge/resolve incidents based on membership permissions.

Indexes and constraints

Use unique constraints for organization membership and service identifiers; indexes for open incidents by organization/service/status; time indexes for timeline events and alert events; foreign-key constraints for integrity; and transaction boundaries for actions such as creating an incident and attaching its first alert.

11. API and Real-Time Design

The frontend should never talk directly to PostgreSQL, Redis or the AI service. It communicates with the core API, which applies authentication, authorization and business rules.

API area	Representative endpoints
Auth	/auth/login, /auth/refresh, /auth/logout, /auth/me
Organizations	/organizations, /organizations/:id/members
Services	/services, /services/:id, /services/:id/health
Telemetry	/ingest/metrics, /ingest/events or collector integration endpoint
Rules	/alert-rules, /alert-rules/:id/test
Incidents	/incidents, /incidents/:id, /incidents/:id/acknowledge, /assign, /resolve
AI	/incidents/:id/ai-analysis, /incidents/:id/postmortem-draft
Analytics	/analytics/reliability, /analytics/incidents
Audit	/audit-events
Integrations	/notification-targets, /telemetry-sources

WebSocket events

- incident.created
- incident.updated
- incident.assigned
- alert.fired
- service.health.changed
- notification.updated

Use WebSockets for state that should appear instantly on an operations dashboard. Normal CRUD/configuration can continue to use REST. The client reconnects safely and refetches authoritative state after reconnect so a dropped WebSocket message does not permanently corrupt the UI.

12. Observability and Telemetry Ingestion

Observability means understanding system behavior from emitted signals such as metrics, logs and traces. OpenTelemetry provides a vendor-neutral way to instrument and collect these signals, while Prometheus is a common time-series metrics system. In AegisOps AI, these concepts become part of the product rather than only developer tooling.

Portfolio-friendly ingestion path

14. Build one or two small demo services that export HTTP request metrics and structured logs.
15. Instrument the core AegisOps API as well, so your monitoring product can monitor itself.
16. Use an OpenTelemetry Collector or simple adapter to normalize incoming signals.
17. Store durable references/summaries in your application database and time-series metrics in Prometheus.
18. Query bounded time windows around an incident for evidence rather than copying unlimited telemetry into PostgreSQL.

Signals you should support

Signal	Examples	How AegisOps uses it
Metrics	request count, 5xx rate, p95 latency, CPU, queue depth	Rules, trends, anomaly scoring.
Logs	structured error message, service, timestamp, correlation ID	Evidence and pattern clues.
Traces	request spans across components	Identify which operation or dependency became slow.
Deploy/change events	version, deployment time, configuration change	Correlate incidents with recent changes.

13. Incident Detection Logic

Rule-based detection

Start with deterministic rules because they are explainable and easy to test. Example: `error_rate > 5%` for 5 minutes, `p95_latency > 1000 ms` for 3 minutes, or `queue_depth > 100` for 10 minutes. A rule should define metric, comparator, threshold, evaluation window, severity, cooldown and target service/environment.

Anomaly-based detection

The ML detector complements rules. It evaluates a metric window and returns an anomaly score. To avoid noisy incidents, require sustained abnormality, combine multiple features, or use a minimum evidence threshold before creating an alert event.

Correlation and deduplication

If three rules fire for the same service during the same failure, you usually want one incident with three attached alerts rather than three unrelated incidents. Build a fingerprint from organization + service + environment + alert family, then use a time window and current open incident state to decide whether to attach or create.

Severity model

Severity	Meaning	Example
SEV-1 / Critical	Major user-facing outage or data-risk condition.	Checkout unavailable for most users.
SEV-2 / High	Significant degradation affecting a meaningful user segment.	Error rate elevated and latency severe.
SEV-3 / Medium	Limited impact or early warning.	Queue backlog growing but service still functional.
SEV-4 / Low	Informational or minor operational concern.	Single non-critical dependency warning.

14. Root-Cause Assistance and AI Reasoning

A strong AI feature starts with evidence retrieval, not with a giant prompt. The core API builds a structured incident context from the database and telemetry queries, removes sensitive values, and sends only the relevant bounded context to the AI service.

Evidence package example

- Incident metadata: service, environment, severity, start time.
- Metric changes: baseline vs incident window.
- Top log patterns around incident start.
- Slow trace/span summaries.
- Recent deployments/configuration changes.
- Previously similar incidents and confirmed causes, if available.
- Human investigation notes.

Expected AI response schema

Field	Meaning
summary	Plain-language description of what is happening.
impact	Observed or estimated user/system effect, clearly labeled.
hypotheses[]	Ranked possible causes.
evidence_refs[]	IDs or labels of evidence supporting each hypothesis.
next_checks[]	Concrete investigation actions.
confidence	Low/medium/high or numeric calibrated indication.

Field	Meaning
uncertainties[]	Information missing or contradictory.

Critical safety rule

The UI must visually distinguish observed facts, AI hypotheses and human-confirmed root cause. This is both good AI design and strong interview material.

15. Security, RBAC and Multi-Tenancy

Security is one of the areas that separates a serious project from a demo. Treat every organization as a tenant boundary and every protected action as a permission decision.

- Password hashing using a modern adaptive password hash.
- Short-lived access tokens or secure server sessions plus safe refresh handling.
- HttpOnly/Secure/SameSite cookies when appropriate.
- Backend guards for organization membership and role permissions.
- Rate limiting for auth, ingestion and AI endpoints.
- Input validation and DTO/schema validation on every external endpoint.
- API key hashing for telemetry source credentials; never store raw secret values after creation when avoidable.
- Audit privileged actions such as role changes, rule edits and integration changes.
- Never expose cross-tenant data through IDs without verifying organization ownership.
- Sensitive log-field redaction before storing or sending to AI.
- CORS/CSRF strategy appropriate to deployment architecture.
- Secret management through environment variables or deployment secret stores, never Git.

Row-level security - optional advanced layer

PostgreSQL supports row-level security policies that can restrict which rows are visible or modifiable. You can use application-level tenant checks as the primary mechanism and optionally demonstrate database-level RLS for selected tables as defense in depth.

16. Background Jobs, Notifications and Reliability

Anything slow, retrievable or scheduled should not block a user request. Examples include notification delivery, periodic alert evaluation, AI analysis, analytics aggregation and cleanup jobs.

Job	Why background processing helps
Alert evaluation	Runs repeatedly without needing a browser request.
Notification delivery	Retries transient email/webhook failures.
AI incident analysis	Can take seconds; UI can show queued/running/completed.
Postmortem generation	Runs after resolution without holding a request open.
Daily reliability rollups	Precomputes analytics for faster dashboards.
Retention cleanup	Removes expired temporary telemetry/evidence according to policy.

Reliability behaviors to implement

- Idempotency for jobs that may retry.
- Exponential backoff for transient failures.
- Maximum retry count and dead-letter/failure state.
- Job correlation IDs for logs.

- Graceful shutdown so workers stop accepting work and finish safely.
- Health/readiness endpoints for API, worker and AI service.

17. Testing Strategy

Your test strategy should prove both code correctness and architecture behavior.

Test layer	What to test
Unit tests	Rule evaluation, severity calculation, correlation fingerprints, RBAC permission functions, anomaly feature builders.
Backend integration tests	Auth, database transactions, tenant isolation, incident workflows, queue enqueueing, API validation.
AI service tests	Feature validation, deterministic response parsing, failure fallback and schema validation.
WebSocket tests	Authorized connection, organization scoping, incident event delivery, reconnect behavior.
Frontend component tests	Incident status controls, permissions, loading/error states and form validation.
End-to-end tests	Login -> service -> rule -> simulated failure -> incident -> acknowledge -> AI analysis -> resolve.
Security tests	Unauthorized/cross-tenant access, invalid tokens, excessive ingestion, unsafe payloads.
Build gates	Typecheck, lint, tests and production build in CI.

The golden demo test

A single automated end-to-end test should be able to create or use a demo service, trigger a controlled failure, wait for an incident, open the incident, verify its evidence, acknowledge it and resolve it. That test proves that the product is integrated rather than a collection of disconnected pages.

18. Deployment and Live Demo Strategy

The final project should have a live URL and a repository that another developer can run locally. Do not require the evaluator to configure ten external services before seeing anything.

Local development

Docker Compose should start PostgreSQL, Redis, the OpenTelemetry/metrics components and optional local supporting services. The web/API/AI services can run in containers or in developer mode depending on your workflow.

Production portfolio deployment

Component	Reasonable deployment option
Web	Vercel or container hosting.
Core API	Railway/Render/Fly/AWS/GCP/Azure container service.
AI service	Container platform with enough memory/CPU for selected model.
PostgreSQL	Managed PostgreSQL.
Redis	Managed Redis.
Metrics stack	Managed service or small container deployment; keep demo retention bounded.
Worker	Separate process/container from API.
CI/CD	GitHub Actions: install -> lint -> typecheck -> tests -> build -> deploy.

Demo mode

Include a seeded demo organization and a "fault simulator" for a demo service. The simulator can intentionally add latency, return 500 errors, overload a queue or emit a dependency failure. This lets recruiters see a real incident appear live instead of waiting for a random failure.

19. MVP vs Advanced Scope

MVP - must be excellent	Advanced - add after MVP works
Auth + organization + RBAC	SSO / OAuth provider integration
Service catalog	Service dependency graph
Metric ingestion + demo service	Full log/trace search experience
Threshold alert rules	Composite rules and maintenance windows
Incident correlation	Cross-service dependency correlation
Real-time incident updates	Mobile/on-call notifications
Isolation Forest anomaly detector	More advanced time-series model comparison
Evidence-grounded AI summary	Similar-incident retrieval / RAG knowledge base
Incident workflow + timeline	Runbooks and approval-gated remediation
Postmortem + analytics	SLO/error-budget module
Docker + CI + live deploy	Kubernetes deployment example

The advanced column is not required to make the project impressive. Finish the MVP at production quality before adding complexity. A complete smaller system is stronger than a huge unfinished architecture.

20. Solo-Developer Build Roadmap

Phase	Outcome
Phase 0 - Specification	Freeze product scope, architecture, conventions, environment assumptions and acceptance criteria.
Phase 1 - Monorepo foundation	Create web, API, AI service, shared packages, Docker dev dependencies, lint/typecheck/test foundations.
Phase 2 - Auth and tenancy	Users, organizations, memberships, RBAC, secure auth, audit basics.
Phase 3 - Service catalog	Services, environments, ownership, health configuration, API/UI.
Phase 4 - Telemetry demo	Demo service, metrics, collector/Prometheus, ingest adapters, health views.
Phase 5 - Alerts	Rule CRUD, evaluator worker, alert events, deduplication.
Phase 6 - Incidents	Correlation, incident state machine, timeline, assignment, acknowledge/resolve.
Phase 7 - Real-time UI	WebSocket gateway, live incident/health updates, reconnect/refetch logic.
Phase 8 - AI anomaly service	Feature extraction, model, scoring endpoint, evaluation and tests.
Phase 9 - AI incident analyst	Evidence package, provider adapter, structured response, guardrails, UI.
Phase 10 - Notifications	In-app + email/webhook, retries and delivery state.
Phase 11 - Analytics/postmortem	MTTA/MTTR, reliability dashboards, postmortem workflow.
Phase 12 - Security hardening	Rate limits, secret handling, tenant tests, audit completeness.
Phase 13 - Quality gate	Unit/integration/e2e, accessibility, performance, error states.
Phase 14 - Productionization	Docker images, CI/CD, migrations, backups, health checks, deployment.
Phase 15 - Portfolio polish	README, architecture diagrams, screenshots, demo data, video walkthrough, CV bullets.

21. How This Benefits a Software Engineering Degree

Software-engineering area	What this project makes you practice
Requirements Engineering	Define actors, use cases, functional/non-functional requirements and scope.
Software Architecture	Choose boundaries, modularity, integration contracts and trade-offs.
Database Engineering	Relational modeling, constraints, migrations, transactions and indexes.
Backend Engineering	APIs, validation, authorization, asynchronous jobs and failure handling.
Frontend Engineering	State management, real-time UX, forms, dashboards and access-aware UI.
Distributed Systems Concepts	Queues, event delivery, retries, idempotency and partial failure.
AI/ML Engineering	Feature pipelines, anomaly detection, model-serving and safe LLM integration.
DevOps	Containers, environments, CI/CD, health checks and deployment.
SRE / Reliability	Metrics, alerts, incident lifecycle, MTTA/MTTR and postmortems.
Software Quality	Unit, integration, end-to-end, security and regression testing.
Security Engineering	RBAC, tenant isolation, secrets, audit trails and rate limits.
Documentation	Architecture decisions, setup instructions, API docs and operational runbooks.

Degree relevance

This project connects multiple subjects that are often taught separately. That is exactly what makes it valuable: you demonstrate that you can integrate software engineering concepts into one functioning system.

22. CV, GitHub and Interview Value

What it can signal to employers

- You can design beyond CRUD.
- You understand production failure and observability concepts.
- You can integrate TypeScript and Python services cleanly.
- You can build real AI features that are measurable and evidence-grounded.
- You understand queues, WebSockets, Docker and CI/CD.
- You think about authorization, tenant isolation and operational safety.
- You can finish and deploy a multi-component system.

Possible CV project entry

AegisOps AI - AI-Powered Incident Management & SRE Platform

Designed and built a multi-tenant incident-management and observability platform using Next.js, NestJS, PostgreSQL, Redis, Python/FastAPI and OpenTelemetry. Implemented real-time incident workflows, alert correlation, background jobs, anomaly detection, evidence-grounded AI triage, RBAC, audit logging, automated testing, Dockerized environments and CI/CD deployment.

What makes the GitHub repository professional

- Clear README with problem, architecture, screenshots and demo URL.

- One-command or documented local startup.
- .env.example with no secrets.
- Database migrations and seed/demo data.
- Architecture Decision Records (ADRs) for important choices.
- OpenAPI/Swagger docs for backend APIs.
- Automated CI status and test instructions.
- Issue-free production build and reproducible Docker setup.
- No fake screenshots that cannot be reproduced from the live product.

23. How to Demonstrate the Project

A recruiter may give you only a few minutes. Design the demo so value appears immediately.

19. Open the live dashboard showing three demo services.
20. Trigger "Checkout latency failure" using the demo simulator.
21. Show the latency/error metrics change and the alert rule fire.
22. Show a new incident appear in real time.
23. Open the incident timeline and attached evidence.
24. Run the AI analysis and show that every hypothesis is tied to evidence.
25. Acknowledge and assign the incident.
26. Stop the fault, resolve the incident and record the confirmed cause.
27. Open the analytics/postmortem view and show the historical record.

A 30-second explanation

Demo pitch

"This is AegisOps AI, a production-style incident-management and observability platform. It ingests operational telemetry, detects rule violations and ML anomalies, correlates related alerts into incidents, updates engineers in real time, and provides an AI analyst that summarizes only the evidence collected around the failure. I built the web app, core backend, Python AI service, PostgreSQL/Redis architecture, background jobs, tests, Docker environment and deployment."

24. Risks, Limitations and Ethical Use

Risk / limitation	How the design handles it
AI hallucination	Outputs are hypotheses with evidence references; human confirms cause.
False-positive anomaly detection	Use thresholds, persistence windows, scoring and evaluation data; do not auto-page on weak signals.
High telemetry volume	Bound retention and use time-series storage rather than transactional DB for every sample.
Secret leakage through logs	Redaction pipeline and sensitive-key filtering before AI requests.
Cross-tenant data leak	Organization-scoped queries, backend authorization tests and optional database RLS.
Notification storm	Deduplication, grouping, cooldowns and retry policies.
Over-engineering	Modular core backend; add distributed components only for clear responsibilities.
Demo dependency cost	Provider abstraction and local/demo fallbacks where practical.

25. Antigravity Prompt Strategy

You should NOT paste one enormous prompt asking Antigravity to build the complete platform in one attempt. Large coding agents are much more reliable when the architecture is frozen first and implementation is delivered in phases with explicit acceptance criteria and verification commands.

How I will give you prompts

Prompt type	What it contains
Master Project Contract	Permanent architecture, coding standards, constraints, security rules, directory conventions and definition of done.
Phase Prompt	One bounded implementation milestone, exact modules/files to inspect, required behavior and acceptance tests.
Verification Prompt	Run lint/typecheck/tests/build/migrations and fix failures without changing intended behavior.
Bug-Fix Prompt	Uses your exact error/log output, asks agent to diagnose root cause, patch minimally and rerun tests.
Hardening Prompt	Security, performance, accessibility, error-state and failure-mode audit after features work.
Release Prompt	Deployment configuration, CI/CD, environment validation and final production checklist.

Prompt size rule

Each phase prompt will be detailed enough that Antigravity does not have to guess architecture, but small enough that you can inspect and test the result before moving on. Normally one prompt corresponds to one major milestone, not one tiny file and not the entire product.

What every implementation prompt will force the agent to do

- Inspect existing workspace before changing code.
- Preserve all previously working functionality.
- Do not leave TODO placeholders, fake API responses or mock data once the real backend exists.
- Keep strict typing and validation.
- Create or update database migrations correctly.
- Write tests for the new behavior.
- Run lint, typecheck, relevant tests and production build.
- Fix failures before claiming completion.
- Summarize files changed, commands run and remaining limitations.
- Never silently change architecture, dependencies or public API contracts without justification.

When you send results back to me

After each phase, send me Antigravity's completion summary and any terminal errors. I will use that as the state of the project and give you the next prompt. This creates a controlled engineering loop rather than blind prompt chaining.

26. Definition of Done

AegisOps AI is not "done" because the pages look good. The portfolio project is done when all of the following are true:

- Live deployed web application is reachable.
- Core incident workflow works end to end with real persisted data.
- Telemetry from a demo service produces visible health signals.
- A controlled failure can produce an alert and incident.
- Real-time updates work without manual refresh.

- AI anomaly scoring works on actual metric windows, not hard-coded labels.
- AI incident analysis is grounded in stored/retrieved evidence.
- RBAC and organization isolation are enforced in backend tests.
- Background jobs have retry/failure behavior.
- PostgreSQL migrations can build a fresh database.
- Docker-based local environment is documented and reproducible.
- Unit, integration and end-to-end tests pass.
- Lint, typecheck and production builds pass.
- CI runs automatically on pushes/pull requests.
- Secrets are excluded from repository.
- README explains architecture, setup, demo and limitations.
- Repository contains meaningful commit history.
- A short demo video or GIF is available.
- CV description matches what the deployed project actually does.

27. Example Interview Questions You Should Be Able to Answer

Question	Strong answer direction
Why NestJS instead of Express?	Because the project benefits from modules, DI, guards, validation, WebSockets and structured testing conventions; Express would work, but NestJS gives a stronger architecture for this size.
Why a separate Python AI service?	The core business domain remains in TypeScript, while Python isolates ML dependencies and model-serving concerns behind a small API.
Why Redis if PostgreSQL already exists?	PostgreSQL stores durable business truth; Redis handles transient queue/caching/event coordination where low-latency ephemeral state is more appropriate.
Why WebSockets?	Incident status and service health should update operators immediately. REST remains authoritative and the client refetches after reconnect.
How do you avoid AI hallucination?	The model receives bounded retrieved evidence, outputs structured hypotheses with evidence references, exposes uncertainty and requires human confirmation for root cause.
How do you avoid alert storms?	Grouping, deduplication fingerprints, evaluation windows, cooldowns, severity rules and incident correlation.
Why not use microservices everywhere?	The expected solo-project scale does not justify the operational cost. A modular monolith plus separate AI/worker processes gives clean boundaries without unnecessary network complexity.
How would you scale it later?	Separate ingestion/evaluation workloads, partition telemetry, use dedicated event streaming, scale WebSocket gateways with shared event backing, and extract high-load modules behind stable contracts.
How do you test tenant isolation?	Integration tests create two organizations and verify that IDs from one tenant cannot be read or modified using credentials from the other.
What is the difference between a fact and an AI hypothesis in your UI?	Facts come from observed telemetry and human-confirmed records; AI hypotheses are clearly labeled, linked to evidence and never stored as confirmed cause until a human action does so.

28. Glossary

Term	Meaning
SRE	Site Reliability Engineering - applying software engineering to reliability and operations.
Observability	Ability to understand system state from outputs such as metrics, traces and logs.
Metric	Numeric time-series measurement, e.g., requests/sec or error rate.
Log	Timestamped event record produced by software.
Trace	End-to-end representation of one request across operations/services.
Span	One timed operation inside a trace.
Alert	A detected condition that requires attention or evaluation.
Incident	A managed operational problem that may contain one or more alerts.
MTTA	Mean Time To Acknowledge.
MTTR	Mean Time To Resolve/Recover, depending on convention.
RBAC	Role-Based Access Control.
Tenant	Independent organization/workspace sharing the same SaaS application.
Idempotency	Repeating an operation does not create unintended duplicate effects.
Backoff	Increasing delay between retry attempts after failures.
Anomaly Detection	Identifying observations that differ meaningfully from expected patterns.
RCA	Root Cause Analysis.
LLM	Large Language Model.
RAG	Retrieval-Augmented Generation; retrieve relevant evidence before generating an answer.
CI/CD	Continuous Integration / Continuous Delivery or Deployment.
OpenTelemetry	Vendor-neutral framework for generating, collecting and exporting telemetry.

29. Reference Sources

The project design is original portfolio architecture. The following official documentation supports key technology and observability concepts used in the blueprint:

Source	URL	Used for
OpenTelemetry - Observability Primer	https://opentelemetry.io/docs/concepts/observability-primer/	Observability and telemetry signals.
OpenTelemetry - What is OpenTelemetry?	https://opentelemetry.io/docs/what-is-opentelemetry/	Vendor-neutral instrumentation and collection.
Prometheus - Alerting Overview	https://prometheus.io/docs/alerting/latest/overview/	Alerting rules and Alertmanager workflow.
Prometheus - Alertmanager	https://prometheus.io/docs/alerting/latest/alertmanager/	Grouping, routing, silencing and inhibition.
NestJS Documentation	https://docs.nestjs.com/	Backend modules, security, queues, WebSockets and microservices capabilities.
NestJS WebSocket Gateways	https://docs.nestjs.com/websockets/gateways	Real-time gateway model.
FastAPI Background Tasks	https://fastapi.tiangolo.com/tutorial/background-tasks/	Background-task concepts and caveats.
PostgreSQL Row Security Policies	https://www.postgresql.org/docs/17/ddl-rowsecurity.html	Optional tenant defense-in-depth.
Redis Streams Documentation	https://redis.io/docs/latest/develop/data-types/streams/	Consumer groups and event-stream primitives.
Redis Pub/Sub Documentation	https://redis.io/docs/latest/develop/use-cases/pub-sub/	Event fan-out and real-time update use cases.

Source	URL	Used for
Docker Compose	https://docs.docker.com/compose	Multi-container application configuration.
GitHub Actions - Understanding Actions	https://docs.github.com/en/actions/get-started/understand-github-actions	Automated build, test and deployment workflows.

Final Perspective

AegisOps AI should be treated as a software product, not an assignment. The value comes from the integration of reliable backend design, real-time incident workflows, observable systems, AI that is tied to evidence, secure multi-tenant behavior, automated testing and a deployment that another person can actually use. If you build the phases in order and refuse to accept mock-only shortcuts, this can become a flagship portfolio project for software-engineering, full-stack, backend, DevOps/SRE and AI-enabled software roles.

Your next implementation step

Create the repository and execute the Master Project Contract / Phase 0 prompt. Do not begin feature coding before the architecture, conventions and acceptance criteria are frozen.